Guida passo-passo: robot a due ruote in equilibrio

Pipeline standard NVIDIA (FreeCAD → URDF → Isaac Lab → RL)

Progetto robot_slalom

Aggiornata al 24 ottobre 2026

Scopo di questa guida

Documenta, passo per passo e in ordine cronologico, la pipeline standard NVIDIA/Isaac Lab usata in questo progetto: dal modello CAD fino all'addestramento con reinforcement learning. Viene aggiornata ad ogni passo compiuto. La regola generale della pipeline (cosa è "standard" e cosa no) è in `PIPELINE_STANDARD_NVIDIA.md`.

Indice

1 Punto di partenza: dal CAD Fusion a FreeCAD **1**

2 Installazione FreeCAD e del server MCP **1**

3 Installazione del workbench CROSS (export URDF) **2**

4 I concetti: Robot, Link, Joint **3**

5 Composizione del robot in CROSS **3**
- 5.1 Struttura . . . 3
- 5.2 Pezzi CAD in ogni link . . . 4
- 5.3 Giunti . . . 4
- 5.4 Come si crea dall'editor . . . 4
- 5.5 Problemi incontrati . . . 5

6 Calcolo di masse e inerzie **5**
- 6.1 Perché non il pulsante di CROSS . . . 5
- 6.2 Masse dei pezzi . . . 5
- 6.3 Lo script . . . 6

7 Il formato URDF **7**
- 7.1 Struttura del file . . . 7
- 7.2 Un link . . . 7
- 7.3 Un joint . . . 8

8 Prossimi passi (da fare) **9**

1 Punto di partenza: dal CAD Fusion a FreeCAD

Il modello del robot a due ruote è stato progettato in **Fusion 360** (CAD): corpo, motori, ruote, batteria, elettronica (ESP32-S3, sensori ToF, camera). Da qui si parte seguendo alla lettera la

pipeline standard NVIDIA, usando **FreeCAD** al posto di Fusion 360 per il passaggio successivo, perché ha un esportatore URDF nativo tramite il workbench **CROSS**:

1. **Importazione**: il modello viene esportato da Fusion in formato neutro (`STEP`, file `export\robot_2ruote_esp32_varD.step`) e importato in FreeCAD.
2. **CROSS**: dentro FreeCAD si definiscono i link e i giunti del robot (corpo, ruota sx, ruota dx, giunti rivoluti attorno all'asse delle ruote) e si assegnano materiali/densità ai pezzi.
3. **Esportazione**: CROSS esporta direttamente `robot.urdf`, con masse, inerzie e mesh incluse.
4. **Simulazione**: Isaac Sim importa l'URDF (`sim_utils.UrdfFileCfg`) e lo converte in USD.
5. **Isaac Lab**: `ArticulationCfg` carica il robot, `DirectRLEnv` definisce l'ambiente RL, `rsl_rl` esegue l'addestramento PPO.

Tutto il codice Python di supporto (configurazione del robot, ambiente RL, script di training, visualizzazione e valutazione) verrà scritto da zero a partire dal nuovo URDF, seguendo solo l'API standard di Isaac Lab, senza bridge o file intermedi scritti a mano.

2 Installazione FreeCAD e del server MCP

Data: 24 ottobre 2026.

1. FreeCAD 1.1 installato in `C:\Program Files\FreeCAD 1.1`.
2. Server MCP `freecad-robust-mcp` (progetto spkane/freecad-addon-robust-mcp-server) installato con pip (Python 3.11, richiesto per compatibilità ABI con FreeCAD):

```
python -m pip install --user freecad-robust-mcp
```

3. Registrato in Claude Code:

```
claude mcp add freecad "C:\Users\aless\AppData\Roaming\Python\Python311\Scripts\
    freecad-mcp.exe" -e FREECAD_MODE=xmlrpc
```

4. Il server richiede la libreria `mcp` versione 1.x (con la 2.x si chiude all'avvio):

```
python -m pip install --user "mcp<2"
```

5. **Workbench dentro FreeCAD**: *Robust MCP Bridge* non è nel catalogo dell'Addon Manager, quindi va installato a mano. Dal repository si copia la cartella `freecad\RobustMCPBridge` in

```
%APPDATA%\FreeCAD\v1-1\Mod\RobustMCPBridge\freecad\RobustMCPBridge\
```

   e si mette un `package.xml` in `...\Mod\RobustMCPBridge\`. Attenzione: FreeCAD 1.1 legge gli addon dalla cartella con la versione (`v1-1\Mod`), non da `FreeCAD\Mod`.

6. Il Python interno di FreeCAD deve avere le dipendenze del bridge (mancava `anyio`):

```
"C:\Program Files\FreeCAD 1.1\bin\python.exe" -m pip install --user freecad-robust-
    mcp
```

7. Avvio: in FreeCAD selezionare il workbench *Robust MCP Bridge* e cliccare *Start MCP Bridge*. Nel Report view compare "MCP Bridge started!" (XML-RPC 9875, socket 9876) e `claude mcp list` mostra `freecad: Connected`.
8. Avvio automatico del bridge a ogni apertura di FreeCAD: in `%APPDATA%\FreeCAD\v1-1\user.cfg`, nel gruppo `RobustMCPBridge`, aggiungere `<FCBool Name="AutoStart" Value="1"/>` (con FreeCAD chiuso, altrimenti viene sovrascritto).

3 Installazione del workbench CROSS (export URDF)

CROSS non è nel catalogo standard dell'Addon Manager (cercando "cross" escono solo *Cross Section* e *Mouse Cross*). Si installa a mano dal repository `github.com/galou/freecad.cross`:

1. Clonare il repository nella cartella Mod, **con il nome esatto** `freecad.cross` (il codice cerca icone, template e finestre in `Mod\freecad.cross\resources`: con un altro nome il workbench si carica ma i template dell'export non vengono trovati):

```
cd %APPDATA%\FreeCAD\v1-1\Mod
git clone --depth 1 https://github.com/galou/freecad.cross.git freecad.cross
```

2. Creare `Mod\freecad.cross\InitGui.py` con una sola riga (l'Addon Manager lo farebbe da sé; col clone manuale va scritto):

```
import freecad.cross.init_gui
```

3. Installare le dipendenze dichiarate in `package.xml` nel Python di FreeCAD:

```
"C:\Program Files\FreeCAD 1.1\bin\python.exe" -m pip install --user urdf-parser-py
    xacro
```

4. Riavviare FreeCAD. Nell'elenco dei workbench compare *CROSS - ROS workbench* con i comandi NewRobot, NewLink, NewJoint, SetMaterial, CalculateMassAndInertia, UrdfExport. L'avviso "`ROS_DISTRO` is not set" è normale: ROS non serve per esportare l'URDF.

4 I concetti: Robot, Link, Joint

Un robot per la simulazione (formato URDF) si descrive con tre soli mattoni, che in CROSS corrispondono ai pulsanti *New Robot*, *New Link*, *New Joint*:

Link = un pezzo rigido. Un insieme di pezzi CAD che si muovono sempre insieme, come se fossero incollati. Per il simulatore è un unico oggetto con massa, baricentro e inerzia. Il link non ha una forma propria: è un'"etichetta" che raggruppa pezzi CAD (per questo nell'albero di FreeCAD, passandoci sopra col mouse, non si evidenzia niente).

Joint = il collegamento tra due link. Dice come un link si muove rispetto all'altro:

- **Parent**: il pezzo "fisso"; **Child**: il pezzo che si muove.
- **Type**: `continuous` (gira all'infinito, come una ruota), `revolute` (gira entro dei limiti, come un gomito), `prismatic` (scorre, come un cassetto), `fixed` (bloccato).
- **Origin**: posizione del perno e direzione dell'asse di rotazione.
- **Effort / Velocity**: coppia (N·m) e velocità (rad/s) massime del motore.

Robot = il contenitore che tiene insieme link e giunti; è ciò che si esporta come `robot.urdf`.

In Isaac Lab la rete neurale comanderà i **giunti** (quanta coppia dare a ogni ruota); il simulatore calcola come si muove il resto in base alle masse dei **link**.

5 Composizione del robot in CROSS

Data: 24 ottobre 2026. File: `export\robot_cross.FCStd` (recuperato dall'autosalvataggio di FreeCAD dopo uno spegnimento del PC, poi salvato con questo nome).

5.1 Struttura

Il robot è fatto di **3 link** e **2 joint**:

- **Link** (pezzi rigidi): `base_link` (il telaio), `ruota_sx` (ruota sinistra), `ruota_dx` (ruota destra). **Anche ogni ruota è un link**: è un pezzo rigido a sé, separato dal telaio, perché si muove (gira) rispetto ad esso.
- **Joint** (collegamenti): `giunto_ruota_sx` collega il telaio alla ruota sinistra, `giunto_ruota_dx` collega il telaio alla ruota destra.

```
              [LINK] base_link  (telaio, 394 g)
                /                       \
  [JOINT] giunto_ruota_sx       [JOINT] giunto_ruota_dx
       (continuous)                  (continuous)
             |                             |
     [LINK] ruota_sx               [LINK] ruota_dx
       (41 g)                        (41 g)
```

In generale: un link per ogni parte che si muove in modo indipendente, un joint tra ogni coppia di link collegati. Ogni joint ha sempre un link *parent* (sopra) e un link *child* (sotto).

Regola usata: tutto ciò che è avvitato al telaio (anche i motori, il cui corpo non gira) va in `base_link`; solo ciò che ruota con l'albero del motore va nel link della ruota.

5.2 Pezzi CAD in ogni link

Lo STEP importato da Fusion produce 40 solidi (`robot_2ruote_esp32_varD000...039`), senza i nomi dei corpi. Le etichette in FreeCAD sono i nomi dei corpi in Fusion: lo STEP mantiene lo stesso ordine, e la corrispondenza è verificata pezzo per pezzo su dimensioni e volumi (es. `esp32_pcb` 69,4 × 25,4 × 1,6 mm, i ToF 13 × 25 mm).

`base_link` (36 pezzi, 394,1 g, baricentro 3,6 mm avanti e 27,8 mm sopra l'asse delle ruote, cioè $z = 60{,}3$ mm):

- Struttura: piastra_alta, piastra_bassa, colonnina_1...4, portabatterie, appoggio_anteriore, appoggio_posteriore.
- Motori e staffe: motore_sx, motore_dx, staffa_alta_sx/dx, staffa_base_sx/dx.
- Batteria: cella_18650_1/2.
- Elettronica: driver_TB6612, step_down_5V, IMU_MPU6050, esp32_pcb, esp32_modulo_WROOM, esp32_usb_1/2, esp32_header_sx/dx, esp32_supporto_1/2.
- Sensori di distanza: ToF_centro, ToF_sx, ToF_dx.
- Camera: cam_supporto, cam_pcb, cam_modulo_WROOM, cam_obiettivo_base, cam_obiettivo.

`ruota_sx` (41,2 g, centro y=+88, z=32,5 mm): ruota_sx_corpo (cerchio stampato), ruota_sx_gomma (camera d'aria).

`ruota_dx` (41,2 g, centro y=−88, z=32,5 mm): ruota_dx_corpo, ruota_dx_gomma.

5.3 Giunti

Giunto	Tipo	Parent → Child	Origin (mm, rotazione)	Effort / Velocity
`giunto_ruota_sx`	continuous	base_link → ruota_sx	(0, +88, 32,5), roll −90°	0,30 N·m / 29,3 rad/s
`giunto_ruota_dx`	continuous	base_link → ruota_dx	(0, −88, 32,5), roll −90°	0,30 N·m / 29,3 rad/s

La rotazione di $-90°$ porta l'asse z del giunto (asse di rotazione in URDF) lungo l'asse delle ruote. Effort e Velocity sono i valori **tipici, non misurati** del motore JGA25-370 a 6 V: coppia di stallo 0,30 N·m e 280 giri/min a vuoto ($280 \cdot 2\pi/60 = 29{,}3$ rad/s).

5.4 Come si crea dall'editor

1. Workbench *CROSS - ROS workbench*.
2. *New Robot* (`N`,`R`): crea il contenitore `robot`.
3. *New Link* (`N`,`L`) con il robot selezionato; rinominare (F2) in `base_link`. Nel pannello *Dati*, proprietà **Real** → "…" e spuntare i pezzi elencati sopra; **Visual** e **Collision** con gli stessi pezzi.
4. Stessa cosa per `ruota_sx` e `ruota_dx`.
5. *New Joint* (`N`,`J`): nel pannello Dati Type, Parent, Child, Origin, Effort, Velocity come in tabella. Usare *Set placement* per posizionare le mesh.
6. Massa e inerzia di ogni link: con lo script della sezione 6 nella console Python di FreeCAD (non con il pulsante *Calculate mass and inertia*, vedi lì perché).

5.5 Problemi incontrati

- **Si vedono solo frecce**: sui link `ShowVisual` e `ShowReal` (pannello *Vista*) erano `False`. Messi a `True`, e nascosti i 40 pezzi CAD originali per non vederli doppi.
- **Ruote staccate**: i pezzi CAD sono in coordinate assolute, ma il link applicava di nuovo la propria posizione (± 88 mm, $-90°$). Corretto impostando su `ruota_sx/dx` `MountedPlacement` = inverso del `Placement` del link (è ciò che fa *Set placement*).
- **Colori** per riconoscere i link: grigio `base_link`, rosso `ruota_sx`, blu `ruota_dx`. Le frecce blu al centro delle ruote sono gli assi dei giunti (`ShowAxis`, lunghezza 60 mm).
- **Nomi dei pezzi sbagliati**: nella prima sessione i 40 solidi erano stati rinominati a intuito (es. i 3 sensori ToF chiamati `supporto_anteriore_*`, driver/step-down/IMU chiamati `scheda_piccola_*` e `componente_1`). Rinominati con i nomi dei corpi Fusion.
- **Masse di origine non documentata**: masse e inerzie erano state calcolate con densità non scritte da nessuna parte (393 g + 2×42 g). Ricalcolate con lo script della sezione 6 dalle masse per pezzo stimate in Fusion: ora coincidono con i valori di Fusion.
- **Baricentro e inerzia delle ruote sbagliati nell'URDF**: il primo export dava per `ruota_sx/dx` il baricentro a $(0; \pm 0{,}088; 0{,}0325)$ m *rispetto alla ruota* (cioè in coordinate assolute) e l'inerzia maggiore su `iyy`, mentre nel sistema del link ruota l'asse di rotazione è z. Lo script della sezione 6 esprime baricentro e inerzia nel sistema di ogni link.
- **Export: la finestra "ROS workspace"**: all'avvio di *Export to URDF* CROSS chiede il workspace ROS (non la cartella di uscita!) e avvisa "…does not appear to be a valid ROS workspace". È innocuo: controlla solo `install/setup.bash` di ROS 2 su Linux. Scegliere `G:\robotTest`. La cartella di uscita è solo l'**OutputPath** del robot.
- **Export che non scrive nulla**: se i file esistono già, nella finestra di conferma vanno messi tutti su **Overwrite**; con *Ignore* CROSS non scrive niente (e non accetta una scrittura parziale). Finché quella finestra resta aperta, FreeCAD (e il bridge MCP) è bloccato.

- **Bridge MCP che non parte da solo**: all'avvio compare "`module 'FreeCAD' has no attribute 'GuiUp'`" e l'auto-start fallisce; avviarlo dal workbench *Robust MCP Bridge* con *Start MCP Bridge*.

6 Calcolo di masse e inerzie

6.1 Perché non il pulsante di CROSS

Calculate mass and inertia di CROSS (verificato nel codice) usa **solo il primo pezzo** della lista Real di ogni link e **una sola densità per link**, e lascia l'inerzia negli assi del CAD. Per `base_link` (36 pezzi di materiali diversi) e per le ruote (cerchio + gomma) il risultato sarebbe sbagliato. Per questo i link hanno `MaterialNotCalculate = True`: il pulsante li salta.

6.2 Masse dei pezzi

Il dato di partenza è la massa di ogni pezzo. Per ora sono stime fatte in Fusion; **quando ci sono i pezzi veri si pesano e si correggono questi numeri**.

Pezzo	Massa (g)	Origine della stima
piastra_alta, piastra_bassa	32,60; 32,63	volume × 0,6 g/cm^3 (PLA stampato)
colonnina_1…4	0,74 cad.	volume × 0,6 g/cm^3
staffa_alta/base_sx/dx	7,39 cad.	volume × 0,6 g/cm^3
portabatterie	9,34	volume × 0,6 g/cm^3
appoggio_anteriore / posteriore	1,27; 0,97	volume × 0,6 g/cm^3
motore_sx, motore_dx	70 cad.	motore equivalente JGA25-370 (**da pesare**)
cella_18650_1/2	46 cad.	datasheet (dato reale)
driver_TB6612 / step_down_5V / IMU_MPU6050	5 / 10 / 3	stima
ToF_centro/sx/dx	3 cad.	stima
esp32_pcb / esp32_modulo_WROOM	8 / 3	stima
esp32_usb_1/2, esp32_header_sx/dx	0,5 / 1 cad.	stima
esp32_supporto_1/2	0,13 cad.	volume × 0,6 g/cm^3
cam_supporto / cam_pcb / cam_modulo_WROOM	0,54 / 5 / 3	stima
cam_obiettivo_base / cam_obiettivo	1 / 2	stima
ruota_sx/dx_corpo	37,06 cad.	volume × 0,6 g/cm^3
ruota_sx/dx_gomma	4,11 cad.	volume × 0,95 g/cm^3 (gomma)
Totale robot	**476,5**	base_link 394,1 + 2 ruote × 41,2

6.3 Lo script

In FreeCAD: *Visualizza* → *Pannelli* → *Console Python*, incollare lo script e premere Invio. Per ogni link: somma le masse dei pezzi, calcola il baricentro, somma le inerzie di ogni pezzo (dalla sua forma, a densità uniforme) portandole sul baricentro comune (teorema di Huygens-Steiner), poi riporta baricentro e inerzia nel sistema del link (per le ruote: quello del giunto). Infine scrive `Mass`, `CenterOfMass`, `Ixx`…`Izz` nei link. Dopo, salvare e rifare l'export URDF.

```
import FreeCAD

# Massa di ogni pezzo in grammi (stime da Fusion 360: da sostituire con i pesi veri).
MASSE_G = {
    "piastra_alta": 32.60, "piastra_bassa": 32.63,
    "colonnina_1": 0.74, "colonnina_2": 0.74, "colonnina_3": 0.74, "colonnina_4": 0.74,
    "staffa_alta_sx": 7.39, "staffa_base_sx": 7.39, "staffa_alta_dx": 7.39, "staffa_base_dx": 7.39,
    "portabatterie": 9.34, "appoggio_anteriore": 1.27, "appoggio_posteriore": 0.97,
    "motore_sx": 70.0, "motore_dx": 70.0,
    "cella_18650_1": 46.0, "cella_18650_2": 46.0,
    "driver_TB6612": 5.0, "step_down_5V": 10.0, "IMU_MPU6050": 3.0,
    "ToF_centro": 3.0, "ToF_sx": 3.0, "ToF_dx": 3.0,
    "esp32_supporto_1": 0.13, "esp32_supporto_2": 0.13, "esp32_pcb": 8.0, "esp32_modulo_WROOM": 3.0,
    "esp32_usb_1": 0.5, "esp32_usb_2": 0.5, "esp32_header_sx": 1.0, "esp32_header_dx": 1.0,
    "cam_supporto": 0.54, "cam_pcb": 5.0, "cam_modulo_WROOM": 3.0,
```

```
    "cam_obiettivo_base": 1.0, "cam_obiettivo": 2.0,
    "ruota_sx_corpo": 37.06, "ruota_dx_corpo": 37.06,
    "ruota_sx_gomma": 4.11, "ruota_dx_gomma": 4.11,
}

doc = FreeCAD.ActiveDocument
robot = [o for o in doc.Objects if getattr(o, "_Type", "") == "Cross::Robot"][0]
links = [o for o in robot.Group if getattr(o, "_Type", "") == "Cross::Link"]
joints = [o for o in robot.Group if getattr(o, "_Type", "") == "Cross::Joint"]

for link in links:
    # Sistema del link: origine del giunto di cui e' figlio (base_link: origine del robot).
    frame = FreeCAD.Placement()
    for j in joints:
        if j.Child == link.Name:
            frame = j.Origin

    m_tot, com = 0.0, FreeCAD.Vector()
    for p in link.Real:
        m = MASSE_G[p.Label] / 1000.0
        m_tot += m
        com += p.Shape.CenterOfMass * m
    com = com * (1.0 / m_tot)

    # Tensore d'inerzia attorno al baricentro del link, assi del CAD (kg*m^2).
    I = [[0.0] * 3 for _ in range(3)]
    for p in link.Real:
        m = MASSE_G[p.Label] / 1000.0
        k = m / p.Shape.Volume * 1e-6
        M = p.Shape.MatrixOfInertia
        Ip = [[M.A11, M.A12, M.A13], [M.A21, M.A22, M.A23], [M.A31, M.A32, M.A33]]
        r = (p.Shape.CenterOfMass - com) * 1e-3
        rv, r2 = [r.x, r.y, r.z], r.x**2 + r.y**2 + r.z**2
        for a in range(3):
            for b in range(3):
                I[a][b] += Ip[a][b] * k + m * ((r2 if a == b else 0.0) - rv[a] * rv[b])

    # Riporta baricentro e inerzia nel sistema del link.
    R = frame.Rotation.toMatrix()
    Rm = [[R.A11, R.A12, R.A13], [R.A21, R.A22, R.A23], [R.A31, R.A32, R.A33]]
    IL = [[sum(Rm[a][i] * I[a][b] * Rm[b][j] for a in range(3) for b in range(3))
           for j in range(3)] for i in range(3)]

    link.MaterialNotCalculate = True
    link.Mass = m_tot
    link.CenterOfMass = FreeCAD.Placement(frame.inverse().multVec(com), FreeCAD.Rotation())
    link.Ixx, link.Iyy, link.Izz = IL[0][0], IL[1][1], IL[2][2]
    link.Ixy, link.Ixz, link.Iyz = IL[0][1], IL[0][2], IL[1][2]
    print("%-10s massa %.1f g  baricentro %s mm  Ixx %.3e Iyy %.3e Izz %.3e"
          % (link.Label, m_tot * 1000, link.CenterOfMass.Base, link.Ixx, link.Iyy, link.Izz))

doc.recompute()
```

Risultato (uguale ai valori calcolati da Fusion 360 sul modello originale):

```
base_link  massa 394.1 g  baricentro (3.6, -0.02, 60.3) mm  Ixx 9.033e-04 Iyy 5.547e-04 Izz 8.051e-04
ruota_sx   massa 41.2 g   baricentro (0, 0, 0) mm          Ixx 1.265e-05 Iyy 1.265e-05 Izz 2.257e-05
ruota_dx   massa 41.2 g   baricentro (0, 0, 0) mm          Ixx 1.265e-05 Iyy 1.265e-05 Izz 2.257e-05
```

Nota: lo script assume che il padre di ogni giunto sia `base_link` con posizione zero (vero per questo robot); con catene più lunghe andrebbero composte le posizioni dei giunti.

7 Il formato URDF

URDF (*Unified Robot Description Format*) è un file di testo in formato XML che descrive un robot come albero di **link** collegati da **joint**. Nato in ROS, è letto da quasi tutti i simulatori, tra cui Isaac Sim. Il nostro è `robot_description\urdf\robot.urdf`.

Unità di misura: sempre SI. Lunghezze in metri, angoli in radianti, masse in kg, inerzie in kg·m^2 (CROSS converte dai millimetri di FreeCAD).

7.1 Struttura del file

```
<robot name="robot">
  <link name="base_link"> ... </link>
  <link name="ruota_sx">  ... </link>
  <link name="ruota_dx">  ... </link>
  <joint name="giunto_ruota_sx" type="continuous"> ... </joint>
  <joint name="giunto_ruota_dx" type="continuous"> ... </joint>
</robot>
```

7.2 Un link

Ogni `<link>` ha tre parti (esempio reale, `ruota_sx`, accorciato):

```
<link name="ruota_sx">
  <visual>       <!-- come si VEDE: una per ogni pezzo CAD -->
    <origin rpy="1.5708 0 0" xyz="0 0.0325 -0.088"/>
    <geometry>
      <mesh filename="package://robot_description/meshes/robot_cross_ruota_sx_gomma.
    dae"/>
    </geometry>
  </visual>
  <collision>    <!-- forma usata per i CONTATTI (qui uguale alla visual) -->
    ...
  </collision>
  <inertial>     <!-- come si MUOVE: massa, baricentro, inerzia -->
    <mass value="0.0418"/>
    <origin xyz="0 0 0"/>
    <inertia ixx="1.30e-05" iyy="1.30e-05" izz="2.32e-05" ixy="0" ixz="0" iyz="0"/>
  </inertial>
</link>
```

- `<visual>`: solo grafica, non influisce sulla fisica.
- `<collision>`: la forma che tocca il pavimento e gli ostacoli. Può essere la mesh vera o una forma semplice (cilindro, scatola), più veloce da simulare.
- `<inertial>`: massa, posizione del baricentro (`origin`, nel sistema del link) e tensore d'inerzia attorno al baricentro. È ciò che conta per la dinamica.
- `<origin xyz rpy>`: spostamento (m) e rotazione (roll, pitch, yaw in radianti) rispetto al sistema di riferimento del link.
- `package://robot_description/...`: percorso della mesh secondo la convenzione ROS (nome del pacchetto = nome della cartella).

7.3 Un joint

```
<joint name="giunto_ruota_sx" type="continuous">
  <parent link="base_link"/>
  <child  link="ruota_sx"/>
  <origin rpy="-1.5708 0 0" xyz="0 0.088 0.0325"/>
  <axis xyz="0 0 1"/>
</joint>
```

- `parent` / `child`: i due link collegati.

- `origin`: dove sta il perno, nel sistema del parent. Qui al centro della ruota sinistra (y = 88 mm, z = 32,5 mm), ruotato di $-90°$ attorno a x.
- `axis`: asse di rotazione nel sistema del joint. z dopo la rotazione di $-90°$ coincide con l'asse della ruota.
- `<limit effort velocity>`: coppia e velocità massime. Per i giunti `continuous` sono facoltativi e CROSS non li scrive; in Isaac Lab si impostano negli attuatori dell'`ArticulationCfg`.

Sistemi di riferimento: il link radice (`base_link`) definisce il sistema del robot; ogni joint posiziona il sistema del link figlio rispetto al padre. Per questo baricentro e mesh delle ruote sono espressi rispetto al centro della ruota, non rispetto all'origine del robot (vedi il problema del baricentro delle ruote).

8 Prossimi passi (da fare)

1. Quando ci sono i pezzi veri: pesarli, correggere le masse e ricalcolare massa e inerzia.
2. FATTO (24/10/2026): esportare l'URDF da CROSS: selezionare `robot` e premere *Export to URDF*, confermando la scrittura di tutti i file. La destinazione è la proprietà **OutputPath** del robot, impostata a `G:/robotTest/robot_description`: una cartella dedicata solo all'output generato (sovrascritta a ogni export), separata da `robot_sim\` dove andrà il codice Isaac Lab scritto a mano. Il nome segue la convenzione ROS `<nome>_description`; nell'URDF le mesh diventano `package://robot_description/meshes/...`. Servono solo `urdf\` e `meshes\`; `package.xml`, `CMakeLists.txt`, `launch\`, `rviz\` sono file ROS non usati da Isaac.
3. Scrivere da capo `robot_cfg.py` (ArticulationCfg/UrdfFileCfg) e l'ambiente RL (DirectRLEnv) usando i nomi di link/giunti del nuovo URDF.
4. Scrivere da capo gli script di training/visualizzazione/valutazione e confrontare i risultati con quelli precedenti (8,33° di appoggio, reward RL) per verificare l'equivalenza del nuovo modello.